Upright Equilibrium Balance of an Inverted Pendulum using Nonlinear Model Predictive Control

Owen McKenney

This informal write-up describes my understanding of Nonlinear Model Predictive Control (NMPC) and how I applied it to drive an inverted pendulum cart system to an upright equilibrium position. I give explanations of the NMPC procedure and mathematical foundation, the derivation of the system dynamics, and the design of the NMPC for this problem.

Note: This is a working document and is subject to change

1 Nonlinear Model Predictive Control

Model Predictive Control (MPC) is a control method that chooses actions by planning ahead using a model of the system [1,2]. Nonlinear MPC (NMPC) functions the same, but uses a nonlinear system model as opposed to a linear model. Instead of determining control input based only on the current error (such as in Proportional-Integral-Derivative control), NMPC predicts the system's behavior over a short time window for different possible inputs. It then selects the input sequence that best achieves the goal while respecting defined system constraints [1, 2]. At each controller update, NMPC follows a repeating "receding horizon" procedure:

1. Measure the current system state.
2. Use the nonlinear dynamics model to predict system behavior over a finite horizon.
3. Solve an optimization problem to select the input sequence that minimizes an objective cost function while respecting system constraints.
4. Apply the first control input of the optimal sequence to the real system for one time step.
5. Shift the prediction horizon and repeat steps 1–4.

In this project, I apply NMPC to drive an idealized inverted pendulum cart system to an upright equilibrium state.

1.1 NMPC Mathematical Formulation

NMPC chooses control inputs by solving an optimization problem for the system at each time step. An arbitrary system is written in discrete time as

$$x_{k+1} = f_d(x_k, u_k) \tag{1}$$

where x_k is the system state at time step k, u_k is the control input applied during that step, and $f_d()$ is a discrete-time model (in this project, this is obtained by integrating the nonlinear dynamics

of the system over one sample period) [1, 2]. At each time step, NMPC solves for a sequence of future control inputs

$$U = \{u_k, u_{k+1}, ..., u_{k+N-1}\} \tag{2}$$

over a prediction horizon of N steps [1, 2]. The goal is to choose U so that the predicted state trajectory follows the desired behavior while keeping inputs and states within set constraints. A standard NMPC problem looks like

$$\begin{aligned} \min_{u_{k:k+N-1}} \quad & \sum_{i=0}^{N-1} \ell(x_{k+i}, u_{k+i}) + \ell_f(x_{k+N}) \\ \text{s.t.} \quad & x_k = \hat{x}_k, \\ & x_{k+i+1} = f_d(x_{k+i}, u_{k+i}), \qquad i = 0, ..., N-1, \\ & u_{\min} \leq u_{k+i} \leq u_{\max}, \qquad i = 0, ..., N-1, \\ & x_{\min} \leq x_{k+i} \leq x_{\max}, \qquad i = 0, ..., N. \end{aligned} \tag{3}$$

where $\hat{x}_k$ is the measured or estimated state at the current time step [1, 2]. Once a candidate control sequence has been chosen, the model constraint $x_{k+i+1} = f_d(x_{k+i}, u_{k+i})$ propagates the state forward step-by-step, producing a predicted trajectory $x_k, x_{k+1}, ..., x_{k+N}$. Basically, the dynamics constraints ensure that the optimizer cannot choose arbitrary future states and enforce that the predicted motion must be consistent with the physics encoded in $f_d()$.

The objective function in (2) is comprised of two parts. First, the summation uses the stage cost $\ell(x_{k+i}, u_{k+i})$ to measure the performance at each step in the horizon. In this project, the stage cost penalizes behavior such as deviation from the upright pendulum state, large angular velocity, the cart drifting away from the center, and excessive control effort. Second, the terminal term $\ell_f(x_{k+N})$ is an additional penalty applied only to the final predicted state. This term encourages the controller to finish the planned control near the goal by the end of the horizon, rather than temporarily improving cost early in the horizon while ending in an undesirable state.

The inequality constraints encode physical and safety limits for the system. In this project, constraints on u ensure practical actuation of the cart, while constraints on x ensure the cart cannot traverse outside defined bounds. In NMPC, these constraints are handled during the planning, which is desirable because the controller doesn't have to react after the system violates a constraint, but instead optimizes for sequences that avoid violations.

After solving the optimization problem, NMPC applies the first control input so that $u_k = u_k^\star$, and then repeats the entire optimization at the next time step using a new measurement $\hat{x}_{k+1}$. This repeated planning strategy makes NMPC robust to disturbances and modeling error. Even when prediction is imperfect, the controller is able to correct the plan at the next update using new state information.

2 Problem Setup

For this problem, I considered a simple cart-pole system, or inverted pendulum on a cart. The control objective is to move the cart horizontally such that the pendulum is stabilized in an upright position. In this implementation, pendulum angle θ is measured from the vertical down direction, which leads the upright equilibrium to correspond to

$$\theta^\star = \pi, \qquad \dot{\theta}^\star = 0, \qquad x_c^\star = 0, \qquad \dot{x}_c^\star = 0. \tag{4}$$

I also imposed constraints on cart travel and actuation, keeping the cart within position and input bounds

$$x_{\min} \le x_c \le x_{\max}, \qquad |u| \le u_{\max}. \tag{5}$$

3 Dynamics Model

I derived the dynamics of an inverted pendulum–cart system using cart acceleration (rather than force) as the input. This simplifies the derivation by treating the cart acceleration as a prescribed input that causes the pendulum's angular equation of motion to be independent of mass. As such, my model captures the correct angular response but assumes an ideal actuator capable of any acceleration instantaneously. A more realistic system model would likely model a force input and include cart mass, friction, and saturation. This would make the cart acceleration state-dependent and reintroduce the variables needed to apply this control to physical hardware [3, 4].

3.1 Geometry and Kinematics

I define the cart position as $(x_c(t), 0)$ with a massless rod of length ℓ. A bob of mass m is attached to the end of the rod. All variables vary with time t. Angle θ is measured from the vertical down, so the position of the mass at the end of the rod is

$$\mathbf{r}_b = \begin{bmatrix} x_c \\ 0 \end{bmatrix} + \ell \begin{bmatrix} \sin\theta \\ -\cos\theta \end{bmatrix}. \tag{6}$$

I also define the radial and tangential unit vectors, as well as the relations between them as

$$\mathbf{e}_r = \begin{bmatrix} \sin\theta \\ -\cos\theta \end{bmatrix}, \qquad \mathbf{e}_t = \begin{bmatrix} \cos\theta \\ \sin\theta \end{bmatrix}, \qquad \dot{\mathbf{e}}_r = \dot{\theta}\,\mathbf{e}_t, \qquad \dot{\mathbf{e}}_t = -\dot{\theta}\,\mathbf{e}_r. \tag{7}$$

Differentiating (6) using the relationships from (7) results in

$$\dot{\mathbf{r}}_b = \begin{bmatrix} \dot{x}_c \\ 0 \end{bmatrix} + \ell\,\dot{\mathbf{e}}_r = \begin{bmatrix} \dot{x}_c \\ 0 \end{bmatrix} + \ell\,\dot{\theta}\,\mathbf{e}_t, \tag{8}$$

$$\ddot{\mathbf{r}}_b = \begin{bmatrix} \ddot{x}_c \\ 0 \end{bmatrix} + \ell\left(\ddot{\theta}\,\mathbf{e}_t + \dot{\theta}\,\dot{\mathbf{e}}_t\right) = \begin{bmatrix} \ddot{x}_c \\ 0 \end{bmatrix} + \ell\,\ddot{\theta}\,\mathbf{e}_t - \ell\,\dot{\theta}^2\,\mathbf{e}_r. \tag{9}$$

With (9), the bob acceleration is decomposed into the pivot acceleration, tangential acceleration, and centripetal acceleration terms.

3.2 Dynamics using Newton's Second Law

The only forces acting on the mass are gravity and the tension of the rod. Applying these forces using Newton's second law leads to

$$m\ddot{\mathbf{r}}_b = -T\,\mathbf{e}_r + m\begin{bmatrix} 0 \\ -g \end{bmatrix}. \tag{10}$$

To eliminate the unknown tension T, I project (10) onto the tangential direction $\mathbf{e}_t$. Because the tension acts along the rod and $\mathbf{e}_r \cdot \mathbf{e}_t = 0$, the tension contributes no tangential component, leaving only gravity and kinematic terms

$$m\ddot{\mathbf{r}}_b \cdot \mathbf{e}_t = m\begin{bmatrix} 0 \\ -g \end{bmatrix} \cdot \mathbf{e}_t. \tag{11}$$

Using (9) and the definition of $\mathbf{e}_t$ from (7), $\ddot{\mathbf{r}}_b \cdot \mathbf{e}_t$ can be represented as

$$\ddot{\mathbf{r}}_b \cdot \mathbf{e}_t = \begin{bmatrix} \ddot{x}_c \\ 0 \end{bmatrix} \cdot \mathbf{e}_t + \ell\,\ddot{\theta}\,\mathbf{e}_t \cdot \mathbf{e}_t - \ell\,\dot{\theta}^2\,\mathbf{e}_r \cdot \mathbf{e}_t = \ddot{x}_c \cos\theta + \ell\,\ddot{\theta}, \tag{12}$$

with the gravity projection being represented as

$$\begin{bmatrix} 0 \\ -g \end{bmatrix} \cdot \mathbf{e}_t = -g \sin\theta. \tag{13}$$

Substituting in (12) and (13) into (11), while canceling out mass m and solving for $\ddot{\theta}$, results in

$$\ddot{\theta} = -\frac{g}{\ell}\sin\theta - \frac{\ddot{x}_c}{\ell}\cos\theta, \tag{14}$$

which is the nonlinear pendulum equation with a horizontally accelerating joint.

3.3 Acceleration-Input Cart Model

Instead of modeling cart dynamics from forces, I command a horizontal acceleration input u and include a linear drag on cart velocity with

$$\ddot{x}_c = u - c\,\dot{x}_c. \tag{15}$$

where $c > 0$ is a damping coefficient modeling the drag on the cart. The dynamics of the pendulum incorporate gravity and an angular damping coefficient $d > 0$. Substituting (15) into (14) results in

$$\ddot{\theta} = -\frac{g}{\ell}\sin\theta - \frac{u - c\,\dot{x}_c}{\ell}\cos\theta - d\,\dot{\theta}. \tag{16}$$

Equation (15) represents the continuous-time state model that I use for prediction and simulation in this project.

4 NMPC Design and Implementation

Similar predictive-control approaches have been applied to inverted pendulum swing-up and stabilization in prior work [3, 4].

4.1 Discrete-Time Predictor

To convert the continuous inverted pendulum dynamics model into discrete steps, I used a fourth-order Runge-Kutta (RK4) method. RK4 numerically integrates the continuous dynamics over a single time step which creates the f_d needed for NMPC. One RK4 step over update period Δt is represented by

$$\begin{aligned} k_1 &= f(x_k, u_k), \\ k_2 &= f\left(x_k + \frac{\Delta t}{2}k_1,\, u_k\right) \\ k_3 &= f\left(x_k + \frac{\Delta t}{2}k_2,\, u_k\right) \\ k_4 &= f(x_k + \Delta t\, k_3,\, u_k) \\ x_{k+1} &= x_k + \frac{\Delta t}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right). \end{aligned} \tag{17}$$

The resulting discrete map f_d is then used inside the optimizer to drive the pendulum cart over the prediction horizon, so the controller can compare different input sequences and select the one that best balances the pendulum.

4.2 Optimizer

The goal of this project was to implement NMPC, not to write an optimization solver from scratch. For that reason, I used existing libraries to handle the numerical optimization, and focused instead on setting up the NMPC objective, constraints, and model prediction correctly. I used the CasADi Python library to define the NMPC optimization problem (variables, cost, and constraints) and then used IPOPT to solve it. In other words, my code builds the problem, and the solver returns the best control sequence at each time step. At each NMPC update, my code performs the following steps:

1. Create decision variables for the predicted states and control inputs across the horizon.
2. Add equality constraints that enforce the dynamics at each step (using the RK4 discrete-time model).
3. Apply bounds on the cart position and control input so the solution stays within limits.
4. Evaluate the cost function (stage cost summed across the horizon plus a terminal cost).
5. Solve the optimization problem and apply only the first control input.

This approach allowed me to use a reliable solver while still implementing the full NMPC loop and tuning the controller for stable swing-up and balance.

4.3 Cost Function

The cost function determines what behavior the NMPC should prefer. For this project, there is a small complication where the angle of the pendulum repeats every 2π; for example, $\theta = \pi$ and $\theta = -\pi$ describe the same angular position (upright). To avoid this wrap-around issue, I represent the upright error using sine and cosine terms

$$e_{upright}(\theta) = \begin{bmatrix} \sin\theta \\ 1 + \cos\theta \end{bmatrix}. \tag{18}$$

This ensures that $e_upright = 0$ whenever $\theta = -\pi, \pi$, and makes the error change smoothly for all angles. Using this upright error, the stage cost from (3) is defined as

$$\ell(x_k, u_k) = w_\theta e_{\text{upright}}(\theta_k)_2^2 + w_\omega \omega_k^2 + w_x x_{c,k}^2 + w_v v_{c,k}^2 + w_u u_k^2 + w_{\Delta u}(u_k - u_{k-1})^2, \tag{19}$$

where the weights $w_\theta, w_\omega, w_x, w_v, w_u, w_{\Delta u}$ are tunable constants that control how the stage cost function prioritizes the following goals:

- w_θ: upright pendulum
- w_ω: minimal spinning (avoid large ω_k)
- w_x: a centered cart ($x_{c,k} = 0$)
- w_v: minimal cart speed

- w_u: minimal acceleration commands
- $w_{\Delta u}$: minimal rapid changes in acceleration

In practice, these weights were tuned until the system exhibited behavior that achieved these goals in unison. In (19), the $(u_k - u_{k-1})^2$ term encourages smoother control by penalizing rapid changes in the commanded acceleration between consecutive time steps, which reduces jitter near the upright equilibrium position. In addition to the stage cost, I used a terminal cost term from (3) that is defined as

$$\ell_f(x_N) = \alpha\Big(w_\theta e_{\text{upright}}(\theta_N)_2^2 + w_\omega \omega_N^2 + w_x x_{c,N}^2 + w_v v_{c,N}^2\Big). \tag{20}$$

where $\alpha > 0$ is a terminal multiplier. Proper tuning of terms inside this terminal cost term encourages the system to avoid sporadic behavior and prioritize ending the horizon near the goal state. In my implementation, I found that increasing terminal multiplier a was critical for getting the swing-up sequence to work properly.

5 Results

To see a demonstration of this control in action in simulation, please see the video in the GitHub repository. The system was tuned to have minimal swing-up.

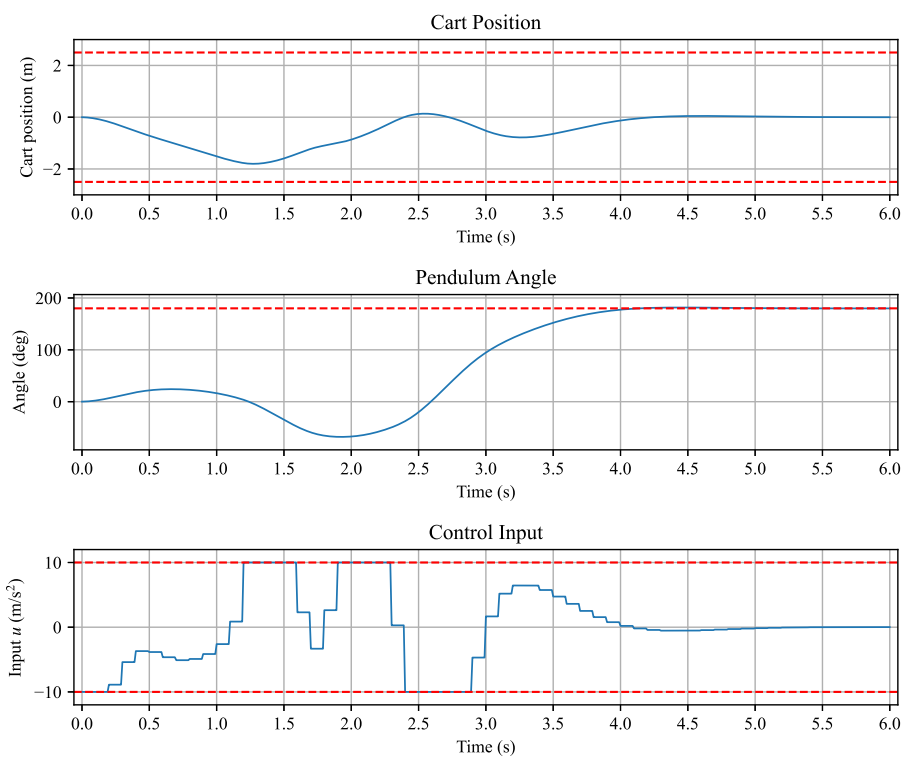

Figure 1: NMPC inverted pendulum swing-up response from demonstration video

6 Future Improvements

Ideally, this control can be tested on actual hardware which introduces much more noise and variability into the system to further test the robustness of the NMPC. With this, I would need to derive new force-input system dynamics to account for critical system variables and make it applicable to a physical system. I would also like to explore a similar implementation in C++ to optimize runtime. Additionally, I would be interested in a new double-pendulum system to apply this same NMPC to solve an upright equilibrium problem. I believe this would prove to be challenging and would force me to re-derive system dynamics and improve computation speeds.

References

[1] L. Grüne and J. Pannek, "Nonlinear Model Predictive Control," in Nonlinear Model Predictive Control, *Communications and Control Engineering. Cham*, Switzerland: Springer, 2017, ch. 3. doi: 10.1007/978-3-319-46024-6_3.

[2] F. Allgöwer, R. Findeisen, and Z. K. Nagy, "Nonlinear model predictive control: From theory to application," *J. Chin. Inst. Chem. Eng.*, vol. 35, no. 3, pp. 299–315, 2004, doi: 10.6967/JCICE.200405.0299.

[3] A. Mills, A. Wills, and B. Ninness, "Nonlinear model predictive control of an inverted pendulum," *Proc. American Control Conf. (ACC)*, St. Louis, MO, USA, 2009, pp. 2335–2340, doi: 10.1109/ACC.2009.5160391.

[4] R. Balan, V. Maties, O. Hancu, and S. Stan, "A predictive control approach for the inverse pendulum on a cart problem," *Proc. IEEE Int. Conf. Mechatronics and Automation (ICMA)*, Niagara Falls, ON, Canada, 2005, vol. 4, pp. 2026–2031, doi: 10.1109/ICMA.2005.1626874.